

[Home](#)[GitHub](#)

CLASSES

[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)[Footer](#)[GET /](#)

backend/src/app.js

```
1.  /**
2.   * @file Fichero principal de la aplicación Express.
3.   * @description Configura y arranca el servidor Express, define middleware
4.   * @requires http-errors
5.   * @requires express
6.   * @requires path
7.   * @requires cookie-parser
8.   * @requires morgan
9.   * @requires cors
10.  * @requires helmet
11.  */
12.
13.  var createError = require('http-errors');
14.  var express = require('express');
15.  var path = require('path');
16.  var cookieParser = require('cookie-parser');
17.  var logger = require('morgan');
18.  var cors = require('cors');
19.  var helmet = require('helmet');
20.
21.  var app = express();
22.
23.  // view engine setup
24.  app.set('views', path.join(__dirname, 'views'));
25.  app.set('view engine', 'pug');
26.
27.  app.use(logger('dev'));
28.  app.use(helmet());
29.
30.  /**
31.   * @name CORS_Configuration
32.   * @description Configuración de CORS para permitir peticiones desde
33.   * @property {string} origin - El origen permitido para las peticiones
34.   * @property {boolean} credentials - Indica si se permiten credenciales
35.   * @property {number} optionsSuccessStatus - Código de estado para peticiones
36.   */
37.  // Configuración de CORS
38.  const corsOptions = {
39.    origin: process.env.FRONTEND_URL || 'http://localhost:3000', //
40.    credentials: true, // Permitir envío de cookies y headers de aut
41.    optionsSuccessStatus: 200
42.  };
43.  app.use(cors(corsOptions));
44.
45.  app.use(express.json());
46.  app.use(express.urlencoded({ extended: false }));
47.  app.use(cookieParser());
48.
49.  const authRouter = require('./routes/auth.routes');
50.  const formularioRouter = require('./routes/formulario.routes');
51.  const registroRouter = require('./routes/registro.routes');
52.  const diarioRouter = require('./routes/diario.routes');
53.  const iaRoutes = require('./routes/ia.routes');
54.  const contactoEmergenciaRouter = require('./routes/contactoEmergencia.routes');
```

```

55.
56. app.use('/api/auth', authRouter);
57. app.use('/api/formulario', formularioRouter);
58. app.use('/api/registro', registroRouter);
59. app.use('/api/registros', registroRouter); // Añadido para aceptar l
60. app.use('/api/diario', diarioRouter);
61. app.use('/api', iaRoutes);
62. app.use('/api/contactos-emergencia', contactoEmergenciaRouter);
63.
64. //app.use('/', indexRouter);
65. //app.use('/users', usersRouter);
66.
67. /**
68.  * @name GET_/
69.  * @description Ruta de prueba para comprobar que el servidor funcio
70.  * @param {object} req - Objeto de petición de Express.
71.  * @param {object} res - Objeto de respuesta de Express.
72.  */
73. app.get('/', (req, res) => {
74.     res.send('Funciona')
75. })
76.
77.
78. /**
79.  * @name 404_Error_Handler
80.  * @description Middleware para capturar errores 404 (Not Found).
81.  * @param {object} req - Objeto de petición de Express.
82.  * @param {object} res - Objeto de respuesta de Express.
83.  * @param {function} next - Función para pasar al siguiente middlewa
84.  */
85. // catch 404 and forward to error handler
86. app.use(function(req, res, next) {
87.     next(createError(404));
88. });
89.
90. /**
91.  * @name Generic_Error_Handler
92.  * @description Middleware para gestionar todos los demás errores.
93.  * @param {object} err - Objeto de error.
94.  * @param {object} req - Objeto de petición de Express.
95.  * @param {object} res - Objeto de respuesta de Express.
96.  * @param {function} next - Función para pasar al siguiente middlewa
97.  */
98. // error handler
99. app.use(function(err, req, res, next) {
100.     // set locals, only providing error in development
101.     res.locals.message = err.message;
102.     res.locals.error = req.app.get('env') === 'development' ? err : {}
103.
104.     // render the error page
105.     res.status(err.status || 500);
106.     res.render('error');
107. });
108.
109. module.exports = app;

```